

Product Requirements Document (PRD)

Urban Furniture — Accounting System

Version: 1.1 **Prepared for:** Hackathon Submission **Document Type:** PRD (Product Requirements Document)

Table of Contents

1. Overview
 2. User Roles & Permissions
 3. Master Data Modules (Detailed Field Specs)
 4. Functional Requirements — Transaction Flows
 5. Reporting Requirements
 6. Non-Functional Requirements
 7. Data Model (High-Level Entity Relationships)
 8. MVP Scope for Hackathon (24–36 hrs)
 9. Success Criteria (Demo Checklist)
 10. Recommended Tech Stack
 11. Assumptions
 12. Open Questions
 13. Appendix A — Database Schema (Plain SQL / PostgreSQL)
-

1. Overview

1.1 Purpose

Urban Furniture needs a lightweight, web-based accounting system to manage core financial operations — recording sales, purchases, and payments, and generating standard financial reports — without relying on a spreadsheet or manual bookkeeping.

1.2 Problem Statement

Small furniture businesses like Urban Furniture currently track sales, purchases, and payments manually or in disconnected spreadsheets, leading to: - Inconsistent or delayed financial reporting - Manual, error-prone debit/credit entry - No real-time visibility into stock, budget, or profitability

1.3 Goal

Build a system where entering a sale or purchase automatically creates the correct underlying accounting entries (debit/credit), keeps ledgers accurate, and produces real-time financial reports (Balance Sheet, P&L, Budget Report) — with zero manual journal math required from the end user.

1.4 Hackathon Scope Note

Given a short build window, the system should prioritize a **correct, complete core loop** (Master Data → Transaction → Journal Entry → Report) over breadth of features (e.g., Analytic Accounts and Budget module can be treated as stretch goals).

2. User Roles & Permissions

Role	Description	Permissions
Admin (Business Owner)	Full system owner	Create / Modify / Archive Master Data, Record Transactions, View Reports
Invoicing User (Accountant)	Day-to-day operator	Create Master Data, Record Transactions, View Reports (no archive rights)
Contact (Customer/Vendor)	External stakeholder	View only their own invoices/bills, make payments
System	Automated engine	Validates data, computes taxes, updates ledgers, auto-generates reports

2.1 Permission Matrix (Detailed)

Action	Admin	Accountant	Contact
Create Contact/Product/CoA/Journal	Yes	Yes	No
Archive Master Data	Yes	No	No
Create Purchase Order / Vendor Bill	Yes	Yes	No
Create Sales Order / Invoice	Yes	Yes	No
Record Payment (own)	Yes	Yes	Own only
View own	Yes	Yes	Yes

Action	Admin	Accountant	Contact
Invoice/Bill			
View All Reports	Yes	Yes	No

3. Master Data Modules (Detailed Field Specs)

3.1 Contact Master

Field	Type	Required	Notes
Name	Text	Yes	
Type	Enum: Customer / Vendor / Both	Yes	Determines available transaction types
Email	Email	Yes	Used for login (Contact role)
Mobile	Text	No	
Address – City	Text	No	
Address – State	Text	No	
Address – Pincode	Text	No	
Profile Image	Image upload	No	

Example records: Vendor – Rahul Sharma; Customer – Nimesh Pathak; Vendor – Azure Furniture

3.2 Product Master

Field	Type	Required	Notes
Product Name	Text	Yes	
Type	Enum: Goods / Service / Combo	Yes	
Sales Price	Decimal	Yes	Used on Sales Order/Invoice
Cost (Purchase Price)	Decimal	Yes	Used on Purchase Order/Bill
Category	Text	No	For grouping/reporting

Example records: Office Chair, Wooden Table, Sofa, Dining Table

3.3 Chart of Accounts (CoA) Master

Field	Type	Required	Notes
Account Name	Text	Yes	e.g., Cash, Bank, Debtors

Field	Type	Required	Notes
Type	Enum: Asset / Liability / Expense / Income / Capital	Yes	Determines report placement

Default seed accounts: - Assets: Cash, Bank, Debtors - Liabilities: Creditors - Income: Sale
Income - Expenses: Purchase Expense

3.4 Journal Master

Field	Type	Required	Notes
Journal Name	Text	Yes	
Type	Enum: Sales / Purchase / Bank / Cash	Yes	
Default Accounts	Reference (CoA)	Yes	Auto-populates journal entries

3.5 Journal Entry (System-Generated)

Field	Type	Required	Notes
Journal	Reference	Yes	Auto-selected based on transaction type
Date	Date	Yes	
Reference	Text	Yes	Links to source Invoice/Bill/Payment
Journal Items	Table (Account, Debit, Credit)	Yes	Must always balance (Total Debit = Total Credit)

3.6 Analytic Account (Stretch Goal)

Field	Type	Notes
Name	Text	e.g., "Project A"
Type	Enum: Income / Expense	For grouping P&L by project/department

3.7 Budget (Stretch Goal)

Field	Type	Notes
Budget Name	Text	
Period	Date range	
Responsible Person	Text	
Planned Amount	Decimal	Compared against actuals in Budget Report

Field	Type	Notes
Analytic Account	Reference	Links budget to a project/department

4. Functional Requirements — Transaction Flows

4.1 Purchase Flow

1. Accountant selects **Vendor**, adds **Product(s)**, **Quantity**, **Unit Price** → creates **Purchase Order**
2. On goods receipt, PO is converted to **Vendor Bill** (captures invoice date, due date)
3. **System auto-generates journal entry:** Debit → Purchase Expense, Credit → Creditors
4. Payment is registered against the bill (Cash or Bank)
5. **System auto-generates journal entry:** Debit → Creditors, Credit → Cash/Bank

4.2 Sales Flow

1. Accountant selects **Customer**, adds **Product(s)**, **Quantity**, **Unit Price**, **Tax** → creates **Sales Order**
2. Sales Order is converted to **Customer Invoice**
3. **System auto-generates journal entry:** Debit → Debtors, Credit → Sale Income
4. Payment is received (Cash or Bank)
5. **System auto-generates journal entry:** Debit → Cash/Bank, Credit → Debtors

4.3 Payment Flow (Standalone)

- Payment can be registered directly against an existing Bill/Invoice
- User selects payment method: Cash or Bank
- System validates the amount does not exceed outstanding balance
- System updates the linked document's payment status (Unpaid / Partially Paid / Paid)

4.4 Core Validation Rules

Rule	Description
Double-entry balance	Every journal entry's total debits must equal total credits
No negative stock (Goods)	Sales Order cannot exceed available stock for physical goods (if inventory tracking enabled)
Payment cap	Payment amount cannot exceed invoice/bill outstanding balance
Mandatory linkage	Every Journal Entry must reference a source document (PO, Bill, SO, Invoice, Payment)
Archive restriction	Master data with existing transactions cannot be hard-deleted, only archived

5. Reporting Requirements

5.1 Balance Sheet

- Real-time snapshot of Assets, Liabilities, and Capital
- Must always satisfy: **Assets = Liabilities + Capital**
- Filter by “as of date”

5.2 Profit & Loss (P&L) Report

- Income (Sale Income) minus Expenses (Purchase Expense + other expenses) = Net Profit
- Filter by date range

5.3 Budget Report (*Stretch Goal*)

- Planned amount vs. Actual amount, grouped by Analytic Account and Period
- Variance (Planned – Actual) highlighted

5.4 Report Generation Flow

1. User selects reporting period (date range)
 2. System aggregates all Journal Entries within that period, grouped by Account Type
 3. Reports render instantly from live ledger data (not cached/static)
-

6. Non-Functional Requirements

Category	Requirement
Performance	Reports should generate in under 2 seconds for typical hackathon-scale data (<5,000 transactions)
Data Integrity	Journal entries are immutable once created; corrections via reversal entries only
Usability	Non-accountant users (Admin) should be able to record a sale/purchase without understanding debit/credit manually
Security	Role-based access control enforced at API level, not just UI level
Auditability	Every transaction stores created-by user and timestamp

7. Data Model (High-Level Entity Relationships)

Contact (1) --< Sales Order / Purchase Order (many)
Product (1) --< Order Line Items (many)
Chart of Accounts (1) --< Journal Entry Line Items (many)
Journal (1) --< Journal Entry (many)
Sales Order (1) -- Customer Invoice (1)
Purchase Order (1) -- Vendor Bill (1)

Invoice/Bill (1) --< Payment (many, until fully paid)
Journal Entry (1) --< Journal Items [Account, Debit, Credit] (many, must balance)

Core tables: contacts, products, chart_of_accounts, journals, journal_entries, journal_items, sales_orders, purchase_orders, invoices, bills, payments, analytic_accounts (stretch), budgets (stretch)

8. MVP Scope for Hackathon (24–36 hrs)

Must-Have (Core Loop)

- Contact, Product, CoA, Journal master data (CRUD)
- Purchase Order → Vendor Bill → Payment flow with auto journal entries
- Sales Order → Customer Invoice → Payment flow with auto journal entries
- Balance Sheet report
- P&L report
- Role-based login (Admin, Accountant, Contact)

Nice-to-Have (If Time Permits)

- Analytic Accounts
- Budget module + Budget Report
- Stock/inventory quantity tracking
- Tax computation on invoices

Out of Scope (For This Hackathon)

- Multi-currency support
 - Multi-company/branch consolidation
 - Bank reconciliation with external bank feeds
 - Recurring invoices
-

9. Success Criteria (Demo Checklist)

1. Create a Vendor and a Customer
 2. Record a full Purchase cycle (PO → Bill → Payment) and confirm correct journal entries were created
 3. Record a full Sales cycle (SO → Invoice → Payment) and confirm correct journal entries were created
 4. Generate Balance Sheet — confirm Assets = Liabilities + Capital
 5. Generate P&L — confirm Net Profit = Income – Expenses matches manual calculation
-

10. Recommended Tech Stack

Note: Prisma has been removed from this recommendation since the team is not familiar with it. The stack below uses plain SQL / a simpler, more beginner-friendly data-access approach instead — no ORM learning curve required.

Option A — Node.js Stack (Recommended for 24–36 hrs)

Layer	Choice	Why
Backend	Node.js + Express	Minimal boilerplate, fastest to scaffold, huge amount of tutorials/StackOverflow support
Database	PostgreSQL	Strong transaction (ACID) support — critical for enforcing “debit = credit” balance rules
Data Access	Plain SQL via pg (node-postgres)	No ORM to learn — you write SQL directly (see Appendix A schema below), full control, and your team can test queries straight in <code>psql</code> before wiring them into code
(Optional, if team wants some abstraction)	Sequelize	If raw SQL feels like too much boilerplate, Sequelize is a more beginner-friendly ORM than Prisma — plain JS objects/classes, huge community, works well with existing SQL schemas
Auth	JWT-based auth with role middleware (Admin/Accountant/Contact)	Simple, no external dependency
Frontend	React + Vite + Tailwind CSS	Fast setup, minimal boilerplate, good for tables/forms-heavy UI
Reports Rendering	Backend computes aggregates (via SQL GROUP BY/SUM) → React renders tables/charts (e.g., Recharts)	Keeps report logic backend-side and easily testable

Layer	Choice	Why
Hosting (demo)	Render / Railway (backend + Postgres) + Vercel (frontend)	Free tier, quick deploy for live demo

Option B — Python Stack (If Team Knows Python Better)

Layer	Choice	Why
Backend	FastAPI	Very fast to write, automatic Swagger API docs — great for hackathon demo
Database	PostgreSQL	Same reasoning as above
Data Access	Plain SQL via psycopg2, or SQLAlchemy Core (not the full ORM layer — just the query-builder part, gentler learning curve than a full ORM)	Keeps things simple while still being safe from SQL injection
Frontend	React + Vite + Tailwind (or server-rendered Jinja2 templates if frontend time is very limited)	

Why PostgreSQL specifically (not MongoDB/NoSQL)

Accounting requires **relational integrity** (a Journal Entry's debits must equal its credits; every entry must trace back to a source document). A relational database with foreign keys and transactions (ACID) enforces this naturally — a document store would require you to hand-roll that integrity logic, costing time you don't have.

Why plain SQL instead of an ORM here

Since the team has no Prisma experience, and there's no time to learn a new ORM's syntax/quirks under hackathon pressure, writing raw SQL (or using a thin query builder) is actually **faster to get right** for a small, well-defined schema like this one. It also makes the core "double-entry balance" logic fully transparent and easy to demo/explain to judges — you can literally show the SQL insert that creates a balanced Journal Entry.

Suggested Build Order (matches MVP checklist)

1. Run the SQL schema (Appendix A) against a fresh PostgreSQL database + seed default accounts
2. Build master data CRUD APIs + minimal UI forms
3. Build Purchase flow (PO → Bill → Payment) with auto journal entry creation logic
4. Build Sales flow (SO → Invoice → Payment) with auto journal entry creation logic
5. Build Balance Sheet & P&L aggregation endpoints + simple table UI

6. Add role-based auth last (wrap existing APIs with middleware) — don't block core logic build on this
-

11. Assumptions

- Single currency, single company (no multi-branch)
 - No physical inventory/warehouse tracking required for MVP (can be added as stretch goal)
 - Tax computation, if included, is a flat percentage per product — not multi-jurisdiction
-

12. Open Questions

- Should Contacts (customers) be able to self-register, or are they only created by Admin/Accountant? (*Spec suggests the latter — created during Contact Master creation.*)
 - Should partial payments be supported, or only full payment per invoice/bill? (*Recommend supporting partial payments — closer to the real workflow and demonstrates payment status logic well.*)
-

Appendix A — Database Schema (Plain SQL / PostgreSQL)

This schema requires **no ORM** — run it directly against a PostgreSQL database using `psql` or any GUI client (e.g., pgAdmin, TablePlus), and your backend can query it with plain SQL (pg in Node, or `psycopg2` in Python).

```
-- =====
-- USERS & ROLES
-- =====
CREATE TABLE users (
  id          SERIAL PRIMARY KEY,
  name        VARCHAR(150) NOT NULL,
  email       VARCHAR(150) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  role        VARCHAR(20) NOT NULL CHECK (role IN ('admin',
'accountant', 'contact')),
  contact_id  INTEGER, -- linked if role = 'contact'
  created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

-- =====
-- MASTER DATA
-- =====
CREATE TABLE contacts (
  id          SERIAL PRIMARY KEY,
  name        VARCHAR(150) NOT NULL,
  type        VARCHAR(10) NOT NULL CHECK (type IN ('customer',
'vendor', 'both')),
```

```

        email            VARCHAR(150),
        mobile           VARCHAR(20),
        address_city     VARCHAR(100),
        address_state    VARCHAR(100),
        address_pincode  VARCHAR(20),
        profile_image    TEXT,
        is_archived      BOOLEAN NOT NULL DEFAULT FALSE,
        created_at       TIMESTAMP NOT NULL DEFAULT NOW()
    );

```

```

ALTER TABLE users
    ADD CONSTRAINT fk_users_contact
    FOREIGN KEY (contact_id) REFERENCES contacts(id);

```

```

CREATE TABLE products (
    id            SERIAL PRIMARY KEY,
    name          VARCHAR(150) NOT NULL,
    type          VARCHAR(10) NOT NULL CHECK (type IN ('goods',
'service', 'combo')),
    sales_price   NUMERIC(12,2) NOT NULL,
    cost_price    NUMERIC(12,2) NOT NULL,
    category      VARCHAR(100),
    is_archived   BOOLEAN NOT NULL DEFAULT FALSE,
    created_at    TIMESTAMP NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE chart_of_accounts (
    id            SERIAL PRIMARY KEY,
    account_name  VARCHAR(150) NOT NULL,
    account_type  VARCHAR(20) NOT NULL CHECK (account_type IN
('asset', 'liability', 'expense', 'income', 'capital')),
    is_archived   BOOLEAN NOT NULL DEFAULT FALSE,
    created_at    TIMESTAMP NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE journals (
    id            SERIAL PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    type          VARCHAR(20) NOT NULL CHECK (type IN ('sales',
'purchase', 'bank', 'cash')),
    default_account_id INTEGER REFERENCES chart_of_accounts(id),
    created_at    TIMESTAMP NOT NULL DEFAULT NOW()
);

```

-- *Stretch-goal master data*

```

CREATE TABLE analytic_accounts (
    id            SERIAL PRIMARY KEY,
    name          VARCHAR(150) NOT NULL,
    type          VARCHAR(10) NOT NULL CHECK (type IN ('income',
'expense')),

```

```

        created_at      TIMESTAMP NOT NULL DEFAULT NOW()
    );

CREATE TABLE budgets (
    id                  SERIAL PRIMARY KEY,
    name                VARCHAR(150) NOT NULL,
    period_start        DATE NOT NULL,
    period_end          DATE NOT NULL,
    responsible_person  VARCHAR(150),
    planned_amount      NUMERIC(14,2) NOT NULL,
    analytic_account_id INTEGER REFERENCES analytic_accounts(id),
    created_at          TIMESTAMP NOT NULL DEFAULT NOW()
);

-- =====
-- PURCHASE FLOW
-- =====

CREATE TABLE purchase_orders (
    id                  SERIAL PRIMARY KEY,
    vendor_id           INTEGER NOT NULL REFERENCES contacts(id),
    order_date          DATE NOT NULL DEFAULT CURRENT_DATE,
    status              VARCHAR(20) NOT NULL DEFAULT 'draft' CHECK (status
IN ('draft', 'confirmed', 'billed', 'cancelled')),
    created_by          INTEGER REFERENCES users(id),
    created_at          TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE purchase_order_items (
    id                  SERIAL PRIMARY KEY,
    purchase_order_id   INTEGER NOT NULL REFERENCES
purchase_orders(id) ON DELETE CASCADE,
    product_id          INTEGER NOT NULL REFERENCES products(id),
    quantity            NUMERIC(10,2) NOT NULL CHECK (quantity > 0),
    unit_price          NUMERIC(12,2) NOT NULL
);

CREATE TABLE vendor_bills (
    id                  SERIAL PRIMARY KEY,
    purchase_order_id   INTEGER NOT NULL REFERENCES
purchase_orders(id),
    vendor_id           INTEGER NOT NULL REFERENCES contacts(id),
    invoice_date        DATE NOT NULL,
    due_date            DATE,
    total_amount        NUMERIC(14,2) NOT NULL,
    payment_status      VARCHAR(20) NOT NULL DEFAULT 'unpaid' CHECK
(payment_status IN ('unpaid', 'partially_paid', 'paid')),
    created_at          TIMESTAMP NOT NULL DEFAULT NOW()
);

-- =====

```

-- SALES FLOW

-- =====

```
CREATE TABLE sales_orders (  
    id SERIAL PRIMARY KEY,  
    customer_id INTEGER NOT NULL REFERENCES contacts(id),  
    order_date DATE NOT NULL DEFAULT CURRENT_DATE,  
    status VARCHAR(20) NOT NULL DEFAULT 'draft' CHECK (status  
IN ('draft', 'confirmed', 'invoiced', 'cancelled')),  
    created_by INTEGER REFERENCES users(id),  
    created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE sales_order_items (  
    id SERIAL PRIMARY KEY,  
    sales_order_id INTEGER NOT NULL REFERENCES sales_orders(id)  
ON DELETE CASCADE,  
    product_id INTEGER NOT NULL REFERENCES products(id),  
    quantity NUMERIC(10,2) NOT NULL CHECK (quantity > 0),  
    unit_price NUMERIC(12,2) NOT NULL,  
    tax_percent NUMERIC(5,2) NOT NULL DEFAULT 0  
);
```

```
CREATE TABLE customer_invoices (  
    id SERIAL PRIMARY KEY,  
    sales_order_id INTEGER NOT NULL REFERENCES sales_orders(id),  
    customer_id INTEGER NOT NULL REFERENCES contacts(id),  
    invoice_date DATE NOT NULL,  
    due_date DATE,  
    total_amount NUMERIC(14,2) NOT NULL,  
    payment_status VARCHAR(20) NOT NULL DEFAULT 'unpaid' CHECK  
(payment_status IN ('unpaid', 'partially_paid', 'paid')),  
    created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

-- =====

-- PAYMENTS

-- =====

```
CREATE TABLE payments (  
    id SERIAL PRIMARY KEY,  
    payment_date DATE NOT NULL DEFAULT CURRENT_DATE,  
    amount NUMERIC(14,2) NOT NULL CHECK (amount > 0),  
    method VARCHAR(10) NOT NULL CHECK (method IN ('cash',  
'bank')),  
    -- Exactly one of the following two should be set  
    vendor_bill_id INTEGER REFERENCES vendor_bills(id),  
    customer_invoice_id INTEGER REFERENCES customer_invoices(id),  
    created_by INTEGER REFERENCES users(id),  
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
    CHECK (  
        (vendor_bill_id IS NOT NULL AND customer_invoice_id IS NULL)
```

```

OR
    (vendor_bill_id IS NULL AND customer_invoice_id IS NOT NULL)
)
);

-- =====
-- JOURNALS & LEDGER (CORE ACCOUNTING ENGINE)
-- =====
CREATE TABLE journal_entries (
    id SERIAL PRIMARY KEY,
    journal_id INTEGER NOT NULL REFERENCES journals(id),
    entry_date DATE NOT NULL DEFAULT CURRENT_DATE,
    reference VARCHAR(150) NOT NULL, -- e.g. "VendorBill#12",
    "CustomerInvoice#7", "Payment#5"
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE journal_items (
    id SERIAL PRIMARY KEY,
    journal_entry_id INTEGER NOT NULL REFERENCES
journal_entries(id) ON DELETE CASCADE,
    account_id INTEGER NOT NULL REFERENCES
chart_of_accounts(id),
    debit NUMERIC(14,2) NOT NULL DEFAULT 0 CHECK (debit
>= 0),
    credit NUMERIC(14,2) NOT NULL DEFAULT 0 CHECK (credit
>= 0),
    CHECK (
        (debit > 0 AND credit = 0) OR (credit > 0 AND debit = 0)
    )
);

-- =====
-- HELPFUL INDEXES
-- =====
CREATE INDEX idx_journal_items_account ON journal_items(account_id);
CREATE INDEX idx_journal_items_entry ON
journal_items(journal_entry_id);
CREATE INDEX idx_vendor_bills_vendor ON vendor_bills(vendor_id);
CREATE INDEX idx_customer_invoices_customer ON
customer_invoices(customer_id);

-- =====
-- SEED DEFAULT CHART OF ACCOUNTS
-- =====
INSERT INTO chart_of_accounts (account_name, account_type) VALUES
    ('Cash', 'asset'),
    ('Bank', 'asset'),
    ('Debtors', 'asset'),
    ('Creditors', 'liability'),

```

```

('Sale Income', 'income'),
('Purchase Expense', 'expense');

-- =====
-- SEED DEFAULT JOURNALS (run after chart_of_accounts insert above)
-- =====
INSERT INTO journals (name, type, default_account_id)
SELECT 'Sales Journal', 'sales', id FROM chart_of_accounts WHERE
account_name = 'Sale Income';

INSERT INTO journals (name, type, default_account_id)
SELECT 'Purchase Journal', 'purchase', id FROM chart_of_accounts WHERE
account_name = 'Purchase Expense';

INSERT INTO journals (name, type, default_account_id)
SELECT 'Bank Journal', 'bank', id FROM chart_of_accounts WHERE
account_name = 'Bank';

INSERT INTO journals (name, type, default_account_id)
SELECT 'Cash Journal', 'cash', id FROM chart_of_accounts WHERE
account_name = 'Cash';

```

A.1 Example — Balanced Journal Entry (Sale)

When a Customer Invoice for ₹5,000 is created, the backend should insert:

```

-- 1. Create the journal entry header
INSERT INTO journal_entries (journal_id, entry_date, reference)
VALUES (1, CURRENT_DATE, 'CustomerInvoice#7')
RETURNING id; -- assume this returns id = 101

-- 2. Insert the two balanced lines (Debit Debtors, Credit Sale
Income)
INSERT INTO journal_items (journal_entry_id, account_id, debit,
credit) VALUES
    (101, (SELECT id FROM chart_of_accounts WHERE account_name =
'Sale Income'), 5000, 0),
    (101, (SELECT id FROM chart_of_accounts WHERE account_name = 'Debtors'), 0, 5000);

```

A.2 Example — Balance Sheet Query

```

SELECT
    coa.account_type,
    coa.account_name,
    COALESCE(SUM(ji.debit), 0) - COALESCE(SUM(ji.credit), 0) AS
balance
FROM chart_of_accounts coa
LEFT JOIN journal_items ji ON ji.account_id = coa.id
LEFT JOIN journal_entries je ON je.id = ji.journal_entry_id
WHERE coa.account_type IN ('asset', 'liability', 'capital')

```

```
    AND (je.entry_date <= :as_of_date OR je.entry_date IS NULL)
GROUP BY coa.account_type, coa.account_name
ORDER BY coa.account_type, coa.account_name;
```

A.3 Example — Profit & Loss Query

```
SELECT
    coa.account_type,
    coa.account_name,
    COALESCE(SUM(ji.credit), 0) - COALESCE(SUM(ji.debit), 0) AS amount
FROM chart_of_accounts coa
LEFT JOIN journal_items ji ON ji.account_id = coa.id
LEFT JOIN journal_entries je ON je.id = ji.journal_entry_id
WHERE coa.account_type IN ('income', 'expense')
    AND je.entry_date BETWEEN :period_start AND :period_end
GROUP BY coa.account_type, coa.account_name
ORDER BY coa.account_type, coa.account_name;
```